NIALL CASSIDY

EVALUATION

A2 WJEC COMPUTER SCIENCE

NIALL CASSIDY CANNDIDATE NO: 0109
CENTER NO: 71741

Section 9: Evaluation (15/100) marks

- produced a detailed evaluation of the effectiveness of the programming language and justified the tools and techniques used
- compared and contrasted the completed system with similar commercially available systems
- identified good features and shortcomings of the completed solution and described significant potential improvements that could be made to improve its effectiveness
- evaluated their own strengths and weaknesses in the design and development of the system
- described specific changes of approach that would be adopted in future to avoid problems experienced during the project
- produced a well-structured and clearly expressed review. Specialist terms have been used with ease and accuracy. Work is error free

9.1 Python Evaluation

The decision to develop Mulligan Motors' second-hand auto dealership database solution in Python proved effective for several reasons. As the language that I have been taught for 4 years, python is my strongest language in all aspects. Python accelerated development through its straightforward syntax, readability, and robust ecosystem of modules which are critical for delivering a functional system within project timelines. Below is an analysis of its strengths and challenges in this context:

Python's simplicity enabled the creation of self-documenting code, ensuring clarity for future maintenance, a necessity for a dealership system requiring frequent updates to inventory and customer records. As an interpreted language, Python streamlined debugging by immediately flagging errors with precise details, reducing troubleshooting time during testing phases.

The SQLite3 module was indispensable for structuring Mulligan Motors' relational database. Leveraging my prior SQL knowledge, SQLite3 simplified and facilitated tasks such as:

- Creating tables for vehicles (e.g., make, model, price, VIN) and customer transactions.
- Enforcing data integrity through column-specific datatypes (e.g., ensuring primary key values are numeric).
- Establishing relationships between tables (e.g., linking sales records to specific inventory items).
- Executing CRUD (Create, Read, Update, Delete) operations for real-time inventory updates.
  SQLite's portability and human-readable syntax ensured the database could be easily interpreted by non-Python developers, future-proofing the system for team collaboration.

While Python lacks native GUI support, TKinter provided a functional, if imperfect, framework for building interfaces. For Mulligan Motors, this included dashboards for staff to view inventory, update sales, or search customer records. Key benefits of TKinter included intuitive button-to-function mapping (e.g., "Add Vehicle" triggering SQLite insertions) and basic form validation to prevent invalid data entry. Its ease of use and compact coding syntax made it almost a requirement to use to meet the time constraints of the solution. Consequently, its role in this project could even be considered mandatory for constructing a professional and aesthetically pleasing finish to the database, which was a direct requirement requested by the boss of Mulligan Motors, Damien Mulligan.

However, TKinter's limitations became apparent in element positioning. Achieving consistent layouts demanded significant trial-and-error, particularly for grid-based designs. This was a significant drawback especially as my home screen became more developed and complicated. In a certain case, I had to revamp the design and format of my main menu buttons as they began to go off screen, even in full screen mode. While languages like C# (with WPF) or Java (with Swing) offer more polished GUI tools, adopting them would have extended development time, an impractical trade-off given the project deadline. Additionally, some would consider TKinter's style of GUI, 'outdated' and 'utilitarian'. Whilst I agree with these sentiments, this is not a customer facing system so aesthetics, while preferential, are not a key element and priority in my system. The last limitation I had with TKinter was its performance. For larger dealerships with massive datasets, you would find compiled languages (e.g., Java, C++) could improve query speeds due to their efficiency and faster overall execution speed. However, SQLite3 and python as an interpreter is not built for such large-scale operations and if Mulligan Motor's ever reached the same capacity they would need a more advanced solution. However, after my investigation into the company, I reasoned that the efficiency offered by python sufficed for Mulligan Motors' mid-scale operations

Python's versatility, paired with SQLite3 and TKinter, delivered a cost-effective solution tailored to Mulligan Motors' operational needs. While languages like JavaScript or C# might offer superior GUI capabilities or performance optimizations, Python's rapid development cycle, extensive documentation, and my existing proficiency made it the optimal choice under time constraints. The final system balances functionality with maintainability, ensuring the dealership can manage inventory, sales, and customer data reliably, a testament to Python's suitability for small-to-medium business solutions.

9.2 Comparison to other systems

To assess the effectiveness of Mulligan Motors' database system, a comparative framework is applied, benchmarking its design and functionality against industry-standard commercial solutions. Below is a reorganized and expanded evaluation, including new insights tailored to the dealership's operational context.

Finding information to compare commercial systems was difficult, as they are almost always proprietary and have measures taken in place to prevent unauthorised access and copying. Whilst there is some information online about the systems of retail giants such as McDonalds, systems which are more akin to Mulligan Motors, mid-scale and non-customer facing, are rare to be documented. The information I have gathered is from such articles and my own experience working in a large retail shop.

1. Database Architecture: Simplicity vs. Scalability

Commercial systems often employ highly normalized databases (beyond Third Normal Form) to minimize redundancy and enhance scalability for large enterprises. However, for Mulligan Motors, a mid-sized dealership with moderate inventory turnover, 3NF normalization proved practical. This approach:

- Simplified table relationships (e.g., linking vehicles to sales records).
- Reduced complexity for staff managing daily updates.
- Avoided over-engineering, aligning with the dealership's immediate needs. *Limitation*: Scaling to accommodate nationwide franchises might require revisiting normalization strategies.
- Overall, the choice to scale to only 3rd Normal Form was an informed decision that weighted Mulligan Motor's needs with the project's time constraints as well as my own knowledge and capability to go beyond 3rd NF.

2. Data Integrity: Foundational Validation

Like commercial inventory systems, Mulligan Motors' solution incorporates core validation checks to ensure accuracy:

- Type checks: Preventing non-numeric entries in fields like *price* or *StaffID*.
- Lookup checks: Drop-down menus for vehicle conditions (e.g., *Used*, *Certified Pre-Owned*).
- Format checks: Validating dates numbers or customer emails.

Although, there is still a technological gap in my validation against advanced validations employed by commercialised systems(e.g., real-time VIN autocomplete, integration with DMV databases) are common in commercial tools but absent here, leaving room for manual entry errors.

3. Security: Balancing Accessibility and Protection

While commercial systems prioritize multi-layered security, Mulligan Motors' solution focuses on essentials:

- Role-based access: Sales vs. admin staff permissions.
- Login authentication: Basic user credential checks.

Proprietary systems would also find it necessary for more security features particularly data encryption, this is a critical gap for protecting sensitive customer information (e.g., payment details). Anyone with access to the file structure that

Mulligan Motors solution is stored in can open the database file and access the the ‘access table’ with ease and all the credentials to log on to the system. Commercial tools often pair encryption with audit trails, which would benefit future iterations which would be beneficial to prevent disgruntled employees from sabotaging the system secretly.

4. User Experience: Efficiency Over Aesthetics

The system prioritizes functionality, mirroring commercial tools in core workflows (e.g., adding inventory, processing sales). Key contrasts would include:

- Navigation: A menu-driven interface requires more clicks for tasks like filtering luxury vehicles, whereas commercial GUIs use shortcuts or predictive search.
- Design: Minimalist layouts lack visual aids (e.g., vehicle thumbnails, color-coded status alerts), which are standard in polished commercial UIs. While aesthetics are secondary for internal staff, a cluttered layout could slow down high-volume transactions during peak sales periods.

To align with industry benchmarks, enhancements could include:

- **Cloud integration**: Enabling multi-branch inventory sync (e.g., if Mulligan Motors expands).
- **Dynamic UI elements**: Embedding interactive dashboards for sales trends, stock aging reports, or employee notices.
- **Automated backups**: Critical for disaster recovery, a standard feature in commercial systems.

To conclude the comparison, Mulligan Motors’ system delivers robust core functionality for its scale, mirroring commercial solutions in data management and validation. However, gaps in encryption, UI intuitiveness, and scalability highlight opportunities for growth. By adopting incremental upgrades, such as encryption protocols and cloud integration, the system can evolve to meet expanding demands while retaining its cost-effective Python foundation. This balance of practicality and ambition ensures the solution remains both viable and competitive in the automotive resale market.

9.3 Strong Features of the solution

Overall, I believe my project effectively met all defined requirements and was delivered within the planned timeframe, which I consider a significant achievement. Three key aspects of the system stand out as particularly successful.

A feature I am proud of is the **search** functionality, which allows users to query *all attributes* within a selected database entity. This flexibility ensures that no data point is excluded from searches, empowering users to locate records based on any criteria they deem relevant. By dynamically constructing SQL queries tied to user input, the system adapts seamlessly to search requests without requiring predefined filters. This design not only enhances usability but also ensures consistency in results. Additionally, integrating feedback messages to display search outcomes, whether a single record, multiple matches, or no results, improved the user experience by providing clear, immediate responses.



Figure 1: The search function

The structure of the **main menus and the utility contained within them** was another strength of the system. Through careful design using Tkinter, I created compact yet highly functional menus that prioritized ease of navigation. Buttons were logically grouped and labelled to guide users effortlessly between tasks such as data entry, reporting, and account management. This simplicity reduced clutter and eliminated the risk of users feeling overwhelmed, while still providing quick access to all core features. The use of consistent visual styling across menus further reinforced usability, ensuring that even first-time users could interact with the system confidently. All the more, the utility that I was able to include in these menus, such as generate report, backup, and the help menu added a degree of professionalism and overall feeling of completeness to the digital system. It automates simple tasks effectively and would save Mulligan Motor employees time doing mundane tasks. The result is increased productivity in employees which can transfer to cost-savings in labour to Mulligan Motors.



Figure 3: The 'admins' access Menu



Figure 2 The 'sales' access Menu

Finally, the **login system** proved to be both secure and efficient. By integrating SQLite3 for credential storage and Tkinter for the interface, I developed a login screen that authenticates users against passwords stored in an ‘access’ table while enforcing role-based access controls. For instance, administrative privileges were restricted to manager accounts, preventing unauthorized modifications to sensitive database settings. Real-time validation checks, such as detecting incorrect passwords, were implemented using try/except blocks, ensuring immediate user feedback without compromising system stability. This approach not only safeguarded data integrity but also streamlined the login process, balancing security with user convenience. The robustness and the extent of which the login system was tested is all evident on the testing section of the documentation.



9.4 Shortcomings of the solution

While my project met core objectives, there are several areas where the system could be improved to enhance security, robustness, and user experience.

A first notable flaw lies in the account creation screen, which inadvertently allows users to assign themselves admin-level privileges. Though newly created accounts are logged in the access table for auditing, the absence of server-side authorization checks means a user could bypass role restrictions by manually selecting "admin" during registration. This oversight undermines the access control framework, as unauthorized privilege escalation becomes technically feasible. While traces of such accounts would remain in the database, the system's reliance on client-side input validation for role assignment represents a critical security gap that could compromise sensitive data or system settings.

Secondly, Although basic validation was implemented, certain inputs lacked rigorous sanitization or error handling. For instance, fields such as dates or numeric entries occasionally accepted invalid formats (e.g., text in numeric fields), which could lead to database inconsistencies or application crashes. Expanding the use of regex patterns or Tkinter's built-in input constraints (e.g., validatecommand) would have strengthened data integrity. Additionally, edge cases, such as unusually long strings or special characters in names, were not fully tested, leaving potential loopholes in user-submitted data.

The system also currently relies on manual database backups, which introduces risk in the event of data corruption or accidental deletion. Implementing scheduled automatic backups using SQLite3's backup command or Python's schedule library which I have found after researching the idea, would have provided a seamless recovery mechanism. Without this, critical data loss could occur, particularly if the system scales to handle larger datasets or multiple concurrent users.

A recurring usability issue stems from the multi-window interface. Each new form (e.g., search screens, reports) opens as a separate window, creating visual clutter and disrupting workflow continuity. This design choice confuses users, as overlapping windows obscure navigation and make it difficult to track active tasks. Transitioning to a single-window layout with Tkinter frames or tabs would create a more cohesive experience, reducing cognitive load and aligning with modern UI/UX standards. However, I did find the separate windows beneficial in multi-tasking in the different areas of the system allowing quick access. Once a predecessor's window was closed, all the windows that branched on from that window (e.g Main menu → Staff Records) would also close resulting in little security risks from windows accidently being left open for unauthorised access.

9.5 Future improvements

If I had more time to complete and expand my knowledge of coding in my program before handing it over to Mulligan Motors, I would incorporate the following features.

Enhancing Report Generation Capabilities

While the current system allows users to export database records to a basic text file, the reporting functionality could be significantly improved to better serve business and analytical needs. Below I have devised a number of methods of which I could achieve this goal depending on how much time I would have to improve it.

1. Diversifying Output Formats

The existing text file exports, though functional, lack formatting and accessibility. Expanding support to include **PDF, CSV, and HTML** formats would cater to diverse use cases. For example:

- **PDF reports** could mirror professional invoices or branded sales summaries, complete with headers, footers, and Mulligan Motors' logos.
- **CSV files** would enable direct integration with spreadsheet tools like Excel for deeper financial analysis.
- **HTML tables** could facilitate quick web-based previews within the application itself. Libraries like ReportLab (for PDFs) or Python's built-in csv module would streamline this implementation.

2. Introducing Customizable Templates
Currently, reports dump raw data without filtering or structure. Adding **template-driven reports** would allow users to:

- Select specific fields (e.g., *vehicle ID, service cost, date*) instead of exporting entire tables.
- Apply filters (e.g., “Show only repairs exceeding £500” or “Sales in Q3 2023”).
- Include dynamic summaries (e.g., total revenue per category, average repair time). A Tkinter configuration window could let users define filters, sort order, and visual themes before generating the report.

3. Embedding Interactive Previews
Rather than forcing users to open external files, an **in-app preview pane** would display reports directly within the system. For text/HTML, this could be a scrollable Tkinter text widget. For PDFs, integrating a lightweight PDF viewer (via tkPDFViewer or embedded browser elements) would allow quick inspections without leaving the application.

4. Automating Scheduled Reports
Critical business workflows (e.g., end-of-month sales summaries) require automation. Integrating a **scheduling feature** would let administrators configure reports to generate daily, weekly, or monthly, saving them to predefined locations or emailing them to stakeholders. Python’s schedule library or SQLite3 triggers could automate this, reducing manual effort.

5. Statistic footers
Adding statistics as a footer to the end-user reports would also enhance the experience significantly. For example, in a report about orders, I would have liked to display how many orders were made in a particular time frame. However, I lack the current knowledge to achieve this and would require a sizeable time frame to learn and code it into Mulligan Motor’s system.

6. Improving Database-Linked Metadata
Currently, reports lack context about their source. Storing **report metadata** (e.g., generated time, user who ran it, filters applied) in the database would create an audit trail. This could be achieved by adding a reports table linked to user and log tables.

While expanding these features would require careful handling of dependencies (e.g., PDF libraries), they are all technically feasible as Python’s ecosystem provides robust tools to implement them incrementally. Prioritizing PDF exports and templating first would deliver immediate value, followed by visualization and automation.

Preventing Overlapping Test Drive Bookings

A critical improvement to the system would involve enforcing logic to detect and block overlapping test drive reservations, ensuring efficient resource allocation and avoiding scheduling conflicts. The core of this feature would rely on time-interval validation within the database. For each new test drive booking, the system would:

- Check the vehicle’s availability by querying existing bookings for the same vehicle.

- Compare time slots: Using SQLite3's datetime functions, the system would identify overlaps where:

To streamline the booking process, the user interface would integrate real-time validation directly into the Tkinter test drive reservation form. When a user selects a vehicle and proposed time slot, the system would instantly check for overlaps by querying the database, displaying a clear warning (e.g., a red banner or pop-up message) if conflicts arise. This immediate feedback prevents users from submitting invalid bookings and guides them toward available slots. To further simplify scheduling. For accessibility, error messages would use plain language (e.g., "This vehicle is booked from 2:00 PM to 3:30 PM. Please choose another time or vehicle") and include tooltips explaining scheduling rules. By unifying these elements, the interface would reduce administrative errors, enhance transparency, and align with Mulligan Motors' focus on user-friendly design.

Improved GUI Elements

To modernize the interface and improve usability, the system would experiment with dynamic, widget-driven design principles in Tkinter, prioritizing both aesthetics and functionality.

Firstly, the system would introduce interactive calendar and timeline widgets, building on the visual calendar concept. A drag-and-drop timeline (using tkinter.Canvas or custom classes) could display all test drives for a vehicle across a week or month, with overlapping slots auto-highlighted in red and available periods in green. Users could click or drag to adjust bookings directly on the timeline, while real-time validation prevents conflicts. The integration of tkcalendar's DateEntry widgets would simplify date selection, and tooltips could display detailed booking information (e.g., customer name, contact) on hover.

Secondly, dynamic filtering would be enhanced with searchable Combobox widgets (via ttk.Combobox), replacing static dropdowns. For instance, typing "SUV" would display only available SUVs, and selecting a time slot would auto-hide already-booked vehicles. Paired with tkinter.Listbox for multi-select filters (e.g., "Show electric vehicles under £30k"), this would streamline decision-making and reduce interface clutter.

Furthermore, disjointed windows would be replaced with a tabbed notebook layout (using ttk.Notebook). Key workflows like "New Booking," "Manage Vehicles," and "View Invoices" would exist as centralized tabs, each featuring status indicators (e.g., red dots for pending approvals) and context-sensitive buttons (e.g., "Quick Report" visible only in the "Sales" tab).

Additionally, raw data displays would be upgraded using ttk.Treeview tables with sorting, pagination, and conditional formatting (e.g., overdue bookings highlighted in orange). For analytics, interactive charts (via matplotlib or customtkinter) could be embedded directly into the GUI, such as live-updating bar charts showing daily test drive demand.

Generic error messages would also be replaced with animated popovers (custom dialogs using Toplevel windows) that pinpoint invalid fields. For example, hovering over a conflicting time slot might show: "Conflict: Vehicle 5 booked until 3:00 PM." For lengthy processes like report generation, ttk.Progressbar widgets with percentage labels would keep users informed.

Finally, adaptive layouts would be prioritized using tkinter.Frame and grid geometry management. The calendar widget could expand to fill screen space on larger monitors, while buttons and forms reorganize into vertical stacks on smaller devices, ensuring consistency across setups. While some widgets (e.g., drag-and-drop timelines) would require custom subclassing or third-party libraries, Tkinter's flexibility and Python's ecosystem make these enhancements achievable incrementally. Prioritizing the calendar and Combobox upgrades first would deliver immediate usability gains, while iterative testing (e.g., A/B testing button placements) could refine workflows.

9.6 My Own Performance

The development of Mulligan Motors' database system showcased my ability to deliver a functional solution that met core requirements, but it also revealed gaps in foresight, scalability, and user-centered refinement. From a technical standpoint, the system demonstrated competency in Python-SQLite3 integration and Tkinter GUI design, with features like normalized database schemas, role-based logins, and real-time validation for overlapping test drives reflecting a solid grasp of relational database principles. The project was completed within the planned timeframe, and structured task prioritization ensured critical components (e.g., report generation, search functionality) were operational early on.

However, the focus on functionality often came at the expense of polish and long-term robustness. For instance, while the GUI provided necessary controls, its multi-window design created a fragmented user experience—a oversight that could have been mitigated by adopting a tabbed interface earlier in development. Similarly, the initial lack of server-side validation for account creation exposed security naivety, requiring retroactive fixes that consumed time better spent on enhancements like automated backups or report templating.

Technical debt accumulated through rushed decisions, such as hardcoding login details until late or neglecting modular structures in my code, which later complicated debugging and scalability. The reliance on basic text-file reporting, while functional, highlighted a missed opportunity to integrate more versatile formats (e.g., PDFs) earlier, limiting the system's professionalism.

In addition, I had never documented an entire program as thoroughly as I have in this project. In the initial steps (discussion, investigation, even design) I found myself unsure of if what I was laying out in the documents were appropriate enough for the scale of the task. I had considered implementing a 'services' table to track the servicing of cars but decided against it. Looking back, I realise that my current design specification is more than enough for Mulligan Motors staff especially considering the time constraint placed upon me. Documentation initially slowed me down in my work significantly. However, by the time I reached my prototype, I became much more familiar with the system and its requirements and I progressed at a much faster pace than before.

Ultimately, the project highlighted my ability to execute technically complex tasks independently but highlighted the need to balance speed with sustainability. Moving forward, I aim to adopt a more holistic approach: prioritizing security and scalability from the outset as well as embracing modular design to reduce rework. While the system achieved its goals, the process revealed that technical skill must be paired with disciplined planning and user empathy to deliver solutions that endure beyond initial deployment.